Associating Tables with Foreign Keys

(https://github.com/learn-co-curriculum/python-p3-associating-tables-with-foreign-keys) (https://github.com/learn-co-curriculum/python-p3-associating-tables-with-foreign-keys/issues/new)

Learning Goals

- Use primary and foreign keys to create the relations in a relational database.

Key Vocab

- **Primary Key**: a number that uniquely identifies one record in a table.
- **Foreign Key**: a column or group of columns that connects one table to another.
- **Join**: a query that returns related records from multiple tables in a single record.
- **One-to-Many**: a type of relationship between tables where one record in table A is connected to multiple records in table B. **e.g.** One person ordering multiple drinks at a bar.
- **Many-to-Many**: a type of relationship between tables where multiple records in table A are connected to multiple records in table B. **e.g.** Students have many classes and classes have many students.

Introduction

In this lesson, we'll relate data from one table to data from another table using foreign keys.

Relating Tables with Foreign Keys

Let's imagine we're building a blogging platform where many different users can write blogs. Our domain would need a couple of different models to represent these two different objects: we'd need a way to store data about different **posts**, as well as the post's **authors**.

In this domain, you could say that an author **has many** posts. The reciprocal of this would be that a post **belongs to** an author.

Now we need to figure out how we can represent that relationship within the constraints of a relational database.

Recall that in the previous lesson we dealt with a similar example: an employee and managers. In that example, an employee **belongs to** a manager and a manager **has many** employees. How did we establish that relationship? By using primary keys and foreign keys — specifically, by using a foreign key in the employees table to reference the primary key in the managers table.

This particular relationship in SQL is known as a **one-to-many** relationship, and it's among the most common type of relationship you'll encounter! To establish this kind of relationship, the rule is always the same:

- Use a **foreign key** on the table of the record that **belongs to** a record in the other table

For our authors and blogs domain, we might end up modeling our data like so. We'd need an authors table with a primary key (**id**):

	id	name	email
	Filt…	Filter	Filter
1	1	Dan	overreacted.io
2	2	Sarah	sarah.dev

And a posts table with a foreign key (**author_id**) to indicate that it **belongs to** a particular author:

We can visualize these relationships using something known as an **Entity Relationship Diagram** (ERD). Here, you can see how one table relates to another using the foreign key and primary key:

Let's build out our own example together in SQL to get some practice.

Code Along Relating Cats to Owners

Let's say we are creating an app that helps a veterinary office manage the pets it sees as patients and the owners of those pets. Let's say this vet is very particular and specializes in cats. Our app will have a database that has a `cats` table and an `owners` table. We will need a way to relate, or connect, these two tables such that a given cat is associated to its owner and a given owner is associated to the cat (or cats) it owns.

For this exercise, we'll be working with the `pets_database.db` file. In your terminal, create the database by running:

```
$ sqlite3 pets_database.db
```

Our ERD for this domain will look like this when we're done:

For the rest of this code along, you can run the SQL commands in your terminal from the `sqlite3` prompt, or you can open the `pets_database.db` file in DB Browser for SQLite, and run the SQL commands from the "Execute SQL" tab.

Step 1: Creating the Cats Table

Let's start by creating a table with the following statement:

```
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER,
  breed TEXT
);
```

Now, go ahead and insert the following cats into the table:

```
INSERT INTO cats (name, age, breed)
VALUES ("Maru", 3, "Scottish Fold");
```

```
INSERT INTO cats (name, age, breed)
VALUES ("Hana", 1, "Tabby");
```

Step 2: Creating the Owners Table

First, we need to create our owners table. An owner should have an ID that is a primary key integer and a name that is text:

```
CREATE TABLE owners (id INTEGER PRIMARY KEY, name TEXT);
```

Now that we have our owners table, we can add a foreign key column to the pets table.

Step 3: Add Foreign Key to Cats Table

Use the following statement to add this column:

```
ALTER TABLE cats ADD COLUMN owner_id INTEGER;
```

Check your `cats` schema (use `.schema` in `sqlite3`, or the "Database Structure" tab in DB Browser) and you should see the following:

```
CREATE TABLE cats (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER,
  breed TEXT,
  owner_id INTEGER
);
```

Great, now we're ready to associate cats to their owners by creating an owner and assigning that owner's ID to certain cats' `owner_id` column.

Step 4: Associating Cats to Owners

First, let's make a new owner:

```
INSERT INTO owners (name) VALUES ("mugumogu");
```

Check that we did that correctly with the following statement:

```
SELECT * FROM owners;
```

You should see the following:

```
1 | mugumogu
```

Mugumogu is the owner of both Hana and Maru. Let's create our associations:

```
UPDATE cats SET owner_id = 1 WHERE name = "Maru";
UPDATE cats SET owner_id = 1 WHERE name = "Hana";
```

Let's check out our updated `cats` table:

```
SELECT * FROM cats WHERE owner_id = 1;
```

This should return:

```
1 | Maru | 3 | Scottish Fold | 1
2 | Hana | 1 | Tabby         | 1
```

Which Table Gets a Foreign Key Column?

Why did we decide to give our `cats` table the foreign key column and not the `owners` table? Similarly, in the example from the beginning of this exercise, why would we give a `posts` table a foreign key of `author_id` and not the other way around?

Let's look at what would happen if we tried to add cats directly to the `owners` table.

Adding the first cat, "Maru", to the owner "mugumogu" would look something like this:

id	name	cat_id
1	mugumogu	1

So far so good. But what happens when we need to add a second cat, "Hana", to the same owner?

id	name	cat_id1	cat_id2
1	mugumogu	1	2

What if this owner gets *yet another cat?* We'd have to keep growing our table horizontally, potentially forever. That is not efficient, or organized.

We can also think about the relationship between our owners and our cats in the context of a **one-to-many** relationship, also known as a **has many** and **belongs to** relationship. An owner can have many cats, but — at least for the purposes of this example — a cat can only belong to one owner. Similarly, an author can write many posts, but each post was written by, and therefore belongs to, only one author.

▶ *Imagine we have a `museums` table and an `artworks` table. Which table would get a foreign key column?*

Resources

- ***What is a Relational Database? - Google Cloud*** ***(https://cloud.google.com/learn/what-is-a-relational-database)***
- ***Difference between Primary Key and Foreign Key - GeeksforGeeks*** ***(https://www.geeksforgeeks.org/difference-between-primary-key-and-foreign-key/)***
- ***SQL Joins - W3Schools*** ***(https://www.w3schools.com/sql/sql_join.asp)***
- ***Airtable's guide to many-to-many relationships*** ***(https://support.airtable.com/hc/en-us/articles/218734758-Airtable-s-guide-to-many-to-many-relationships)***
- ***Practice SQL Queries on SQLBolt*** ***(http://sqlbolt.com/lesson/select_queries_review)***
- ***dbdiagram.io: ERD Drawing Tool*** ***(https://dbdiagram.io)***